

1. The Business Question

Suppose we have this `employees` table:

<code>employee_id</code>	<code>employee_name</code>	<code>manager_id</code>
1	Ravi	NULL
2	Priya	1
3	Arun	1
4	Meena	2
5	Kiran	2

Business question:

Display each employee along with their manager's name.

Expected result:

<code>employee_name</code>	<code>manager_name</code>
Ravi	NULL
Priya	Ravi
Arun	Ravi
Meena	Priya
Kiran	Priya

2. Why do we need a Self Join?

The important observation is:

The employee and manager are stored in the same table.

For example:

employees

employee_id	employee_name	manager_id
1	Ravi	NULL
2	Priya	1
3	Arun	1
4	Meena	2
5	Kiran	2

For Priya:

Priya's manager_id = 1

And employee 1 is:

Ravi

So we need to match:

employee.manager_id
=
manager.employee_id

3. The Self Join

```
SELECT
  e.employee_name AS employee,
  m.employee_name AS manager
FROM employees e
LEFT JOIN employees m
  ON e.manager_id = m.employee_id;
```

The interesting part is:

```
FROM employees e
LEFT JOIN employees m
```

We are joining the **employees table to itself**.

That's why it is called a:

4. Why Two Aliases?

This is the most important concept to explain to trainees.

We have:

employees e

and:

employees m

They are actually the **same table**.

But we give them different aliases because they represent different roles.

employees e



EMPLOYEE

employees m



MANAGER

So:

e.employee_name

means:

Employee's name

while:

m.employee_name

means:

Manager's name

5. Understand the JOIN condition

The most important line is:

ON e.manager_id = m.employee_id

Read it in English:

The employee's manager ID should match the manager's employee ID.

For example:

Priya

e.employee_id = 2
e.employee_name = Priya
e.manager_id = 1

Now find employee 1 in the second copy of the table:

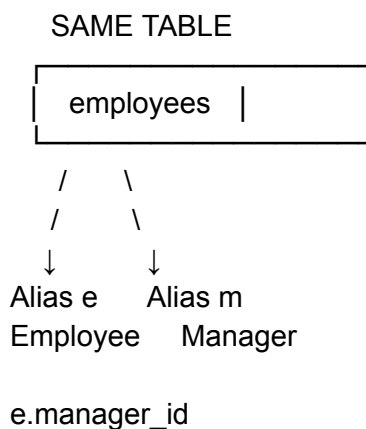
m.employee_id = 1
m.employee_name = Ravi

Therefore:

Priya → Ravi

6. Visual Explanation

Teach it like this:



↓
m.employee_id

The relationship is:

e.manager_id = m.employee_id

7. Why LEFT JOIN?

Suppose Ravi is the CEO:

employee_id = 1
manager_id = NULL

Ravi doesn't have a manager.

If we use:

INNER JOIN

Ravi will disappear because:

NULL = employee_id

doesn't produce a match.

With:

LEFT JOIN

Ravi remains:

employee	manager
Ravi	NULL
Priya	Ravi
Arun	Ravi
Meena	Priya

Kiran Priya

So explain:

Use LEFT JOIN when you also want employees who don't have a manager.

8. If we use INNER JOIN

```
SELECT
  e.employee_name AS employee,
  m.employee_name AS manager
FROM employees e
JOIN employees m
  ON e.manager_id = m.employee_id;
```

Result:

employee e	manager
Priya	Ravi
Arun	Ravi
Meena	Priya
Kiran	Priya

Ravi is missing because Ravi has no manager.

9. Another Useful Self Join Example

You can also teach:

Find employees who report to the same manager.

For example:

Priya → Ravi

Arun → Ravi

Therefore Priya and Arun are colleagues under the same manager.

Query:

```
SELECT
  e1.employee_name AS employee_1,
  e2.employee_name AS employee_2,
  m.employee_name AS manager
FROM employees e1
JOIN employees e2
  ON e1.manager_id = e2.manager_id
JOIN employees m
  ON e1.manager_id = m.employee_id
WHERE e1.employee_id < e2.employee_id;
```

Result:

employee_ 1	employee_ 2	manager
Priya	Arun	Ravi
Meena	Kiran	Priya

The condition:

```
WHERE e1.employee_id < e2.employee_id
```

is important because otherwise we'd get duplicate pairs:

Priya → Arun

Arun → Priya

10. The Simple Rule for Self Join

Give trainees this formula:

SELF JOIN

Same table

+

Different aliases

+

Relationship between two rows

For employee-manager:

employees e

JOIN

employees m

ON e.manager_id = m.employee_id

Ask trainees these 3 questions

Whenever they see a self join, ask:

1. **Why are we using the same table twice?**
2. **What does each alias represent?**
3. **Which columns establish the relationship?**

If they can answer these three questions, they can usually write the self join correctly.